

Tech Stack

Introduction

The proposed AI evaluation system is built using a modern technology stack that supports Retrieval-Augmented Generation (RAG), semantic search, automated evaluation, and scalable web application development. Each technology has been selected based on its performance, ease of integration, community support, and suitability for AI-powered applications.

1. Frontend

Technology

React.js

Purpose

React.js is used to build the user interface of the application. It provides a responsive and interactive web experience where users can upload documents, submit queries, and view AI-generated responses along with evaluation scores.

Why React.js?

- Component-based architecture
 - Fast and responsive UI
 - Easy API integration
 - Large developer community
 - Suitable for modern web applications
-

2. Backend

Technology

Python + Flask

Purpose

The backend manages user requests, processes uploaded documents, communicates with the RAG pipeline, and interacts with the Gemini API.

Why Python + Flask?

- Excellent support for AI and Machine Learning
 - Lightweight and easy to develop
 - Easy integration with LangChain and AI libraries
 - Fast API development
-

3. Database

Technology

SQLite

Purpose

SQLite stores application data such as user requests, generated responses, evaluation scores, and metadata.

Why SQLite?

- Lightweight and serverless
 - Easy to configure
 - Suitable for prototype and academic projects
 - No separate database server required
-

4. Vector Database

Technology

FAISS (Facebook AI Similarity Search)

Purpose

FAISS stores vector embeddings of document chunks and performs semantic similarity searches during document retrieval.

Why FAISS?

- Extremely fast vector search
 - Efficient similarity matching
 - Open-source and lightweight
 - Easy integration with LangChain
-

5. Embedding Model

Technology

Sentence Transformers (all-MiniLM-L6-v2)

Purpose

The embedding model converts document chunks and user queries into dense vector representations for semantic search.

Why all-MiniLM-L6-v2?

- High semantic accuracy
 - Lightweight and fast
 - Low computational requirements
 - Widely used in RAG applications
-

6. Large Language Model (LLM)

Technology

Gemini API

Purpose

Gemini generates context-aware responses using the user's query and the retrieved document context.

Why Gemini API?

- Advanced natural language understanding
 - High-quality response generation
 - Supports long-context processing
 - Easy API integration
 - Suitable for Retrieval-Augmented Generation systems
-

7. Libraries and Frameworks

Library / Framework	Purpose
LangChain	Builds and manages the RAG pipeline

Library / Framework	Purpose
RAGAS	Evaluates retrieval quality and response quality
TruLens	Monitors application performance and evaluation
Sentence Transformers	Generates semantic embeddings
FAISS	Vector similarity search
Flask	Backend API development
React.js	Frontend development
NumPy	Numerical computations
Pandas	Data processing and analysis

Technology Stack Summary

Component	Selected Technology
Frontend	React.js
Backend	Python + Flask
Database	SQLite
Vector Database	FAISS
Embedding Model	Sentence Transformers (all-MiniLM-L6-v2)
Large Language Model	Gemini API
RAG Framework	LangChain
Evaluation Frameworks	RAGAS + TruLens

Conclusion

The selected technology stack provides a robust foundation for developing a reliable AI-powered RAG evaluation system. React.js and Flask enable efficient web application development, while LangChain orchestrates the RAG workflow. Sentence Transformers and FAISS support accurate semantic retrieval, Gemini API generates intelligent responses, and RAGAS together with TruLens ensures comprehensive evaluation and monitoring. This technology stack is scalable, easy to maintain, and aligns with current industry best practices for building trustworthy AI applications.